

Research Summary: PRAC: Round-Efficient 3-Party MPC for Dynamic Data Structures

Paper by: Sajin Sasy, Adithya Vadapalli, Ian Goldberg (2023)

Hitarth Makawana
Roll No: 230479
hitarthkm23@iitk.ac.in

Ronav Puri
Roll No: 230815
ronavgp23@iitk.ac.in

April 2, 2026

Premise

The paper presents **Private Random Access Computations (PRAC)**, a 3-party Secure Multi-Party Computation (MPC) framework designed to support random-access data structure algorithms with efficient communication costs in terms of both rounds and bandwidth. PRAC achieves algorithmic improvements by exploiting the observation that *a secure protocol can gain efficiency if it explicitly reveals information inherently leaked by the algorithm*.

- **Oblivious Binary Search:** An optimized protocol for static data that utilizes *Incremental Distributed Point Functions (IDPFs)* to explicitly leak access patterns. This reduces the bandwidth cost from $\mathcal{O}(\log^2 n)$ to $\mathcal{O}(\log n)$ for obviously searching over n items.
- **Oblivious Heaps:** A restrictively dynamic data structure supporting arbitrary insertions and minimum element deletions. Its implementation leverages optimized binary search to reduce insertion communication rounds from $\mathcal{O}(\log n)$ to $\mathcal{O}(\log \log n)$. Additionally, EXTRACT-MIN's bandwidth is reduced to $\mathcal{O}(\log n)$ using *Wide DPFs (WDPFs)*.
- **Oblivious AVL Trees:** The first oblivious AVL tree protocol for MPC in a distributed trust setting, enabling a fully dynamic data structure. It supports arbitrary insertions and deletions with $\mathcal{O}(\log n)$ rounds and bandwidth, ensuring no party learns the underlying data or the tree's structure.

Distributed Point Functions (DPFs) and Variations

Distributed Point Functions (DPFs)

- **Description:** A DPF is represented as two complete binary trees of height $h = \log n$, where each computing party learns exactly one of the trees.
- **Construction:** Each node within the tree contains a λ -bit label and a single advice/flag bit. The construction begins with the roots of the two trees having random λ -bit labels. An invariant, where corresponding nodes in the two trees are identical except for one on each level, is maintained through the use of h correction words, one for each non-leaf level.
- **Evaluation:** Evaluating the DPF involves traversing the path from the root to a specific leaf i , applying the necessary correction words along the way and outputting the final label on the leaf.

Incremental DPFs (IDPFs)

- **Description:** IDPFs leverage the tree-based construction to allow parties to evaluate their DPFs at bitstrings of any length $1 \leq j \leq \log n$.
- When evaluated, the parties will produce different outputs exactly when the input string is a prefix of the target index i^* , and will output the same value otherwise.
- Although constructing an IDPF costs roughly 50% more than a regular DPF, it is highly efficient because it effectively yields $\log n$ DPFs whose target indices are the prefixes of a single target index.

Wide DPFs (WDPFs)

- **Description:** WDPFs are a variation where the leaf nodes are significantly wider than the internal nodes of the tree.
- For a WDPF of width w , the leaf nodes are exactly w times longer than the internal nodes.
- This is achieved by applying a “length- w -stretching” Pseudorandom Generator (PRG) right before applying the final correction word, which itself is w times longer.
- While highly useful for bulk related-index updates, creating a WDPF of width w requires $n \cdot (w - 1)$ more AES evaluations than a standard DPF.

Distributed Oblivious RAMs (DORAMs)

- **Core Concept:** DORAMs allow multiple computing servers to store and process data collectively without any single server ever knowing the data contents or access patterns.
- Two main categories - *communication-efficient* and *computation-efficient*.

PRAC’s Approach (DUORAM)

- PRAC uses DUORAM (communication-efficient) because it has near-zero initialization costs and stores memory efficiently as secret shares.
- It shifts the heavy lifting like generating DPFs to a preprocessing phase, making live online operations extremely fast (constant rounds/bandwidth).
- PRAC extends the state-of-the-art DUORAM, providing more flexibility in how memory is shared (supporting XOR- or additive-shared indices and memory in any combination).

Oblivious Binary Search

- **Core Premise:** The protocol searches a sorted, additive-shared Distributed ORAM (DORAM) for a specific item. It returns the secret-shared index of the smallest memory location containing a value at least as large as the target item.
- **Basic Approach:** It executes a standard binary search algorithm using general-purpose DORAM reads at each step. It requires $\log n$ separate DORAM read operations, which consequently requires generating $\log n$ independent Distributed Point Functions (DPFs).
- **Inefficiency:** This basic method over-hides information. It completely conceals the accessed indices, even though the structural path of a binary search (jumping left or right by predictable powers of 2) is already public knowledge.

- **Optimized Binary Search (PRAC's Approach):** PRAC explicitly leaking the inherently known structural access pattern of the search. It replaces the $\log n$ standard DPFs with exactly one Incremental DPF (IDPF). The optimization works because the sequence of accessed indices forms an incrementally growing prefix (appending a bit at each depth based on the comparison result), which perfectly matches the structure of an IDPF. Unlike the basic version, the optimized protocol completely eliminates the need for expensive Flag-Word MPC multiplications.
- **Performance Gains:** This algorithmic improvement reduces the communication bandwidth overhead from $\mathcal{O}(\log^2 n)$ to $\mathcal{O}(\log n)$ and lowers computation from $\mathcal{O}(n \log n)$ to $\mathcal{O}(n)$.

Oblivious Heaps

- **Core Concept:** PRAC implements a restrictively dynamic heap data structure. It supports arbitrary insertions (HEAPINSERT) and the deletion of the minimum element at the root (EXTRACTMIN). The heap utilizes an implicit structure where the root is at index 1, the left child is at $2i$, and the right child is at $2i + 1$.

Basic Oblivious Protocols

- **Basic HEAPINSERT:** Adds the new item to the end of the heap array and obliviously compares and swaps it with its parent all the way up to the root to restore the heap property. This requires $\mathcal{O}(\log n)$ sequential oblivious comparisons and swaps, but notably requires zero DORAM operations.
- **Basic EXTRACTMIN:** Saves the root, replaces it with the last leaf node, and trickles it down using a HEAPIFY algorithm. At each level, it obliviously compares the node with its children and swaps it with the smaller one.
- **The Inefficiency:** The basic EXTRACTMIN requires three independent oblivious reads and three updates at each level using a general-purpose DORAM. This is inefficient because it hides the inherently public relationship between the accessed indices (parent, left child, right child).

Optimized Protocols

- **Optimized HEAPINSERT:** The specific path from the root down to the newly added empty node is public knowledge, and this path is already sorted. PRAC uses the optimized binary search protocol (which uses a single IDPF of height $\log \log n$) to quickly find the exact position on this path where the new item belongs. Once the position is found, PRAC uses parallel Flag-Word multiplications to obliviously shift all larger elements down one spot and write the new item into the resulting hole, drastically reducing communication rounds.
- **Optimized EXTRACTMIN:** PRAC explicitly reveals that the algorithm is reading related indices (node and its two children) at each level. It uses a single Wide Incremental DPF to execute these reads and updates simultaneously. Because a WDPF's leaf stores three random values instead of one, it can update the parent, left child and right child in the accessible sets using the exact same DPF.

AVL Trees

- **Explicit Structure:** Unlike heaps and binary search arrays which use an implicit tree structure dictated by memory indices, AVL trees use an explicit structure. Nodes are stored arbitrarily in the DORAM, and the tree structure is maintained dynamically via explicit child pointers to avoid the prohibitive worst-case cost of oblivious reorganization during rebalancing.

- **Oblivious Insertion (Delayed Balancing):** Instead of performing oblivious rotations at all levels of the tree to hide the depth of an imbalance, the AVL insert utilizes a novel recursive insert protocol. This protocol returns an insertion state, st_I , that captures the exact imbalance point (if any) during the insertion. A subsequent FixImb protocol is then executed to perform necessary rotations obliviously and update balance bits, arriving at a balanced state.
- **Oblivious Deletion:** Deletions are processed by obliviously handling three cases: (i) if the node to be deleted has no children, it is deleted directly, (ii) if it has one child, the node is deleted and its parent is updated to point to the only child, and (iii) if it has both children, it is swapped with its successor first and then deleted. Unlike insertion, deleting a node can require imbalance-fixing rotations at each level on the deletion path, requiring FixImb to be performed at every level.

Implementation & Results

The PRAC framework was evaluated with a reference implementation in C++ using Boost Asio, simulated under realistic Internet conditions (30 ms latency, 100 Mbps throughput).

- **Binary Search Performance:** The optimized binary search provides $> 27\times$ improvements in wall-clock time and $> 3\times$ reduction in bandwidth over other approaches for a memory with 2^{26} items.
- **Heap Performance:** For the same 2^{26} item setting, the heap’s Extract-Min protocol achieves a $> 31\times$ speedup in wall-clock time and a $> 13\times$ reduction in bandwidth.
- **Conclusion:** By shifting to an explicit structure and intelligently revealing inherently leaked access patterns, PRAC dramatically improves the viability of complex, dynamic data structures like AVL trees and heaps in 3-party distributed-trust environments.